DSA4213 Assignment 1: Language Models, Embeddings and Representations

Edward Chew · Student ID: A0271710N · Code: https://github.com/EdwardCKB/NLP-for-Data-Science

Comparison question. On a small, single-author corpus, does model quality depend more on the architecture or on what the input embeddings already know? And does better next-token prediction give better sentence representations for a downstream task? The three parts answer this in turn: architecture (Part I), embedding knowledge (Part II) and transfer (Part III).

1. Setup

Corpus (Parts I–II). I used three Jane Austen novels from NLTK's Project Gutenberg collection: *Emma*, *Persuasion* and *Sense and Sensibility*. Each novel is split into chapters, and chapters are assigned contiguously 80/10/10 to train/validation/test, per novel, so no passage crosses splits.

Preprocessing:

- Remove Gutenberg's `_italics_` underscores and lowercase the text. Tokenise with NLTK `sent_tokenize`/`word_tokenize`, keeping punctuation as tokens. Missed sentence-final periods (`knightley.`) are split off, and `<eos>` is appended after each sentence.
- Build the vocabulary from train only with frequency ≥ 3: 4,667 types. Everything else maps to `<unk>`.
- The splits have 340,494 / 56,886 / 50,489 tokens, with `<unk>` rates of 1.9% / 2.7% / 2.8%. All models share this vocabulary, so perplexities are directly comparable.

Hardware and software. Intel i7-13620H, NVIDIA RTX 4060 Laptop GPU (8 GB), PyTorch 2.14.1 (CUDA 12.6), seed 42 everywhere. The five language-model trainings add up to about 13 minutes of GPU time (Tables 1–2); everything runs from `Assignment1.ipynb`.

Evaluation.

- Test cross-entropy (CE) is the mean negative log-likelihood (in nats) of every test token after the first, predicted from the preceding test tokens only. PPL = exp(CE).
- The RNN and LSTM read the test stream as one sequence, carrying their hidden state across 64-token chunks. The Transformer has a fixed 64-token context, so it uses a sliding window with stride 32: every scored token sees ≥ 32 previous tokens.
- Padding never occurs in the contiguous LM streams, but the loss uses `ignore_index=<pad>` throughout, and a sanity-check cell in the notebook tests that padding is excluded.
- A second check tests causality: changing tokens after position t leaves the logits at positions ≤ t unchanged.

2. Part I: comparing language model families

Models.

- **KN trigram:** interpolated Kneser–Ney with a single absolute discount d tuned on validation (d = 0.85).
- **RNN:** 1-layer tanh RNN, hidden size 256.
- **LSTM:** 2 layers, hidden size 256.
- **Transformer:** 4 pre-LayerNorm decoder blocks, d_{model} = 128, 4 heads, FFN 512, learned positions, causal mask, context 64.
- All neural models use 100-d randomly initialised, trainable input embeddings and an untied output layer. Dropout is 0.3 for the RNN and LSTM and 0.2 for the Transformer.

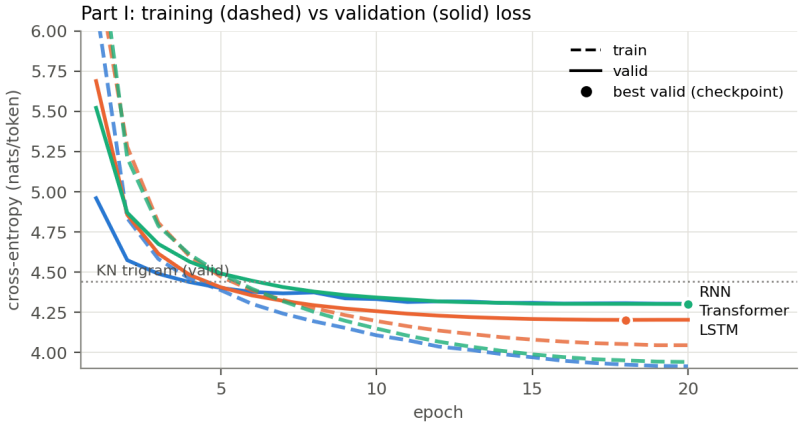
Shared training budget.

- 20 epochs of 32 parallel streams × 64-token chunks (about 166 steps per epoch, 3.3k steps in total).
- AdamW (weight decay 0.01), 200-step warm-up then cosine decay, gradient clipping at 1.0. The checkpoint with the best validation loss is kept.
- The only per-model choice is the peak learning rate, picked from {1e-3, 3e-3} on validation PPL:
  - RNN: 75.8 → 74.0 at 3e-3.
  - LSTM: 79.1 → 66.5 at 3e-3. At 1e-3 the LSTM was still under-fitted when the learning rate decayed.
  - Transformer: 73.9 at 1e-3 vs 74.7 at 3e-3. At 3e-3 it overfitted after epoch 9.

Table 1. Test results and cost. Training time is the median epoch time × 20 (robust to a laptop stall; in this run the raw sum agrees within 3%). Inference is one pass over the test set as 32 streams × 64-token chunks: CPU throughput uses 4 threads and the median of 5 passes, GPU throughput the median of 10 passes after warm-up.

Model	Params	Best epoch	Train time	Test CE	Test PPL	CPU tok/s	GPU tok/s (min–max)
KN trigram	0.30M n-gram entries	–	1.4 s (counting)	4.473	87.6	62k*	–
RNN	1.76M	20	47 s	4.312	74.6	25.2k	0.87M (0.44–1.26M)
LSTM	2.56M	18	136 s	**4.225**	**68.4**	12.6k	88k (70–135k)
Transformer	1.88M	20	195 s	4.302	73.9	18.1k	137k (73–268k)

*The trigram figure is pure-Python dictionary lookups for single-token probabilities. Generation needs the full distribution over the vocabulary and is far slower.



Observations.

- **Prediction quality.**
  - All neural models beat the smoothed trigram, by 13–19 PPL points.
  - The LSTM is clearly best (68.4).
  - The RNN (74.6) and Transformer (73.9) are statistically indistinguishable *from a single seed*.
- **Learning behaviour.**
  - The RNN converges fastest: it passes the trigram's validation loss at epoch 4 (LSTM at epoch 5, Transformer at epoch 7).
  - The RNN and Transformer then show the same pattern. Validation loss flattens around epoch 10 at about 4.30, while training loss keeps falling to 3.92/3.94. The final train–validation gap is 0.39/0.36 nats, so both are **memorising the 340k-token corpus**.
  - Validation loss never rises in a sustained way (the RNN wobbles by at most +0.006 nats), because dropout and the decaying learning rate damp overfitting. The Transformer at lr 3e-3 did overfit (best epoch 9).
  - The LSTM keeps improving until epoch 18 with a gap of only 0.15 nats. It is the best-generalising model, not the best-fitting one.
- **Cost.**
  - On the CPU, throughput ranks RNN > Transformer > LSTM. That follows from per-token work: one tanh recurrence vs two stacked LSTM cells, against parallel attention over 64 positions.
  - On the GPU, repeated passes of the same model spread by 1.9–3.7× (Table 1), and the three architecturally identical LSTMs of Table 2 measured 88k–113k tok/s. At batch 32 these small models are limited by kernel-launch overhead, so only the RNN's large lead (about 6×) is reliable. The Transformer–LSTM gap (1.6×) is within the noise on this laptop.
  - Training-time differences come partly from the implementation. The LSTM is built from two single-layer cuDNN calls, because the fused 2-layer cuDNN LSTM crashed at process exit on this Windows build. It took 6.8 s per epoch (median), against 2.3 s for the single RNN.

Generated text. I used five prompts and generated 30 tokens from each model, with identical decoding: temperature 0.8, top-*k* 40, <unk> banned, the same seeds, and greedy decoding as well. Repetition is measured over 50 sampled continuations.

	KN	RNN	LSTM	Transformer
distinct-2 (sampled)	0.647	0.624	0.599	0.624
repeated-trigram rate (greedy)	0.364	0.450	0.543	0.236

"It was a fine morning, and" — **KN:** she wanted , elinor , " what is to be sure . but i can not you had been most anxious to avoid society to him … **RNN:** she continued , when she had passed of her , as she had left the room , and to be happy to be conveyed into her by the house … **LSTM:** be extremely much mistaken . they were so to make his own account . a man to be made in a hurry from mr. knightley … **Transformer:** she believed , who had no difficulty in a very different room , but it was a great deal of place , and yet knew no doubt of the

drawing-room ... **Greedy:** KN → "the two miss steeles , as she could not be in town , and the two miss steeles ..."; LSTM → "the same time , and the same time , ..."

Observations from the samples:

- **Local fluency.** The trigram is locally plausible but breaks grammar across trigram boundaries ("but i can not you had been"). The neural models produce longer well-formed clauses, and the RNN even opens and closes quotation marks correctly.
- **Consistency.** No model keeps a topic or a set of characters beyond one sentence. All of them mix the novels (Elinor next to Mr. Knightley), because the training stream concatenates them.
- **Repetition.** Every model loops under greedy decoding.
  - The trigram falls into the same loop ("the two miss steeles...") for 4 of the 5 prompts. With a 2-token history, the prompt is forgotten once it enters a high-probability chain.
  - The LSTM, the model with the best PPL, is the most repetitive under greedy decoding and the least diverse under sampling.
  - *Hypothesis:* lower cross-entropy means sharper predictive distributions, so the same temperature and top-*k* produce more mode-seeking text. The Transformer, whose attention can look back directly at its own recent output, loops least.

Trade-offs. *Observed:* the LSTM gives the best perplexity for about 3× the RNN's training time and half its CPU throughput. The Transformer gives no PPL gain over the RNN at this scale, and is the slowest to train here. The trigram is the cheapest by far, but worst by 13–19 PPL points.

Possible explanations, not tested:

- A 340k-token corpus and a 3.3k-step budget favour the LSTM's gating inductive bias over attention, which typically needs more data.
- The carried recurrent state gives the RNN and LSTM an unbounded context at test time, compared with the Transformer's 32–64 tokens.
- Parameter counts are not matched (1.8–2.6M). The LSTM has the most parameters but overfits least, so size alone does not explain its lead.

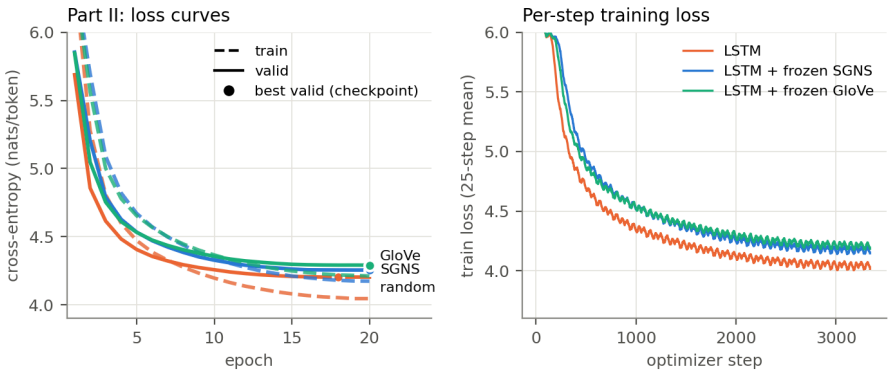
3. Part II: embedding variants (LSTM)

I chose the LSTM because it is the best Part I model. Its Part I run is reused as the random, trainable setting. Two new LSTMs were trained with the same architecture, data, budget and lr (3e-3). In each, the whole 4,667 × 100 input table is frozen (`requires_grad=False`, verified by a sanity check in the notebook).

- **Own SGNS:** skip-gram with negative sampling (gensim; window 5, 10 negatives, 10 epochs) on the train split only, with rare words mapped to `<unk>`. Every vocabulary word is covered. Training the embeddings took 29.4 s on the CPU, reported separately from LM training.
- **Public GloVe 6B-100d:** trained on about 6B tokens of Wikipedia and Gigaword. The dimension of 100 was chosen to match this release.
  - Coverage is 97.75% of types and 94.2% of training tokens. Most of the token gap is `<eos>` (3.7% of tokens) and `<unk>`.
  - The 105 missing words are mostly archaic spellings (*surprized, chuse, shewed*), Austen place names (*uppercross, kellynch, donwell*) and hyphenated compounds (*self-command, good-nature*).
  - Missing rows are drawn once from $N(0, \sigma^2)$, with σ the standard deviation of the GloVe matrix (seed 42), then frozen. `<pad>` is zero.

Table 2. LSTM with three input-embedding settings (same architecture, data and training procedure). LM train time is the median epoch time × 20; on this laptop these times are indicative only.

Embedding	Trainable params	Best epoch	Test PPL	Train / valid CE at best	Gap	Valid CE at epoch 5	Mean grad norm	Step-loss std (last epoch)	LM train time
random, trainable	2.56M	18	**68.4**	4.053 / 4.203	0.15	4.406	0.226	0.076	136 s
own SGNS, frozen	2.09M	20	71.3	4.174 / 4.255	0.08	4.534	0.221	0.073	219 s (+29 s SGNS)
GloVe, frozen	2.09M	20	73.7	4.212 / 4.292	0.08	4.532	0.226	0.073	192 s



Observations.

- **Quality ranking.** Random-trainable (68.4) beats own-SGNS-frozen (71.3), which beats GloVe-frozen (73.7).
- **Convergence.** Both frozen models start more slowly: at epoch 5, validation CE is about 0.13 nats worse. They are still improving when the learning rate reaches zero (best epoch 20). Their *training* loss is also higher, so they **under-fit** rather than generalise better. Their train–valid gap is smaller (0.08 vs 0.15) only because they fit the training data less well.
- **Stability.** Stability is the same across the three settings: equal gradient norms (about 0.22), equal step-to-step loss noise, and no loss spikes.

Interpretation: trainability vs extra data.

- *Trainability* matters most. A table that can adapt to the LM objective beats either fixed table, even though freezing removes only 18% of the parameters.
- Between the two fixed tables, **in-domain beats more data**. Vectors learned from about 340k Austen tokens beat vectors from 6B modern tokens.
- The nearest neighbours show why. GloVe's are semantically clean but reflect modern news senses: *letter* → statement, memo; *happy* → 'm, feel. The SGNS neighbours are noisy but reflect Austen's usage: *letter* → written, note, post; *walk* → afternoon, chaise.
- Next-token prediction on this corpus rewards distributional and syntactic fit to the corpus, which neither fixed table fully provides. GloVe's extra data does **not** pay off for perplexity here. Part III shows where it *does* pay off.

4. Part III: downstream sentiment classification (SST-2)

Data.

- GLUE SST-2 has hidden test labels, so its official **validation set (872 full sentences, 50.9% positive) is my held-out test set**.
- From the training phrases with ≥ 5 words, I sampled 10,000 for training and 2,000 as a development set for choosing C (seed 42).
- Sentences are mapped to the Austen vocabulary, which gives **26% `<unk>`**. Common review words such as *film, movie, fun, funny* and *boring* are all unknown. This domain shift is the dominant factor in this part.

Representations. The language models are frozen and run in eval mode. Right padding is harmless because the models are causal.

- **last:** the top-layer hidden state at the last real token.
- **mean:** the average of top-layer states over the real tokens, excluding padding.

Every representation uses the same classifier: StandardScaler + logistic regression, with $C \in \{0.001, 0.01, 0.1, 1\}$ chosen on the dev set.

As a reference point, I also built a **GloVe average** baseline: the mean of the same frozen GloVe table over the same Austen-vocabulary tokens, with no LM involved. Accuracy confidence intervals come from 2,000 bootstrap resamples of the test set, and differences use a paired bootstrap.

Table 3. SST-2 test results.

Representation	Dim	Test acc. [95% CI]	Macro-F1	LM test PPL
majority class	–	0.509	0.337	–
LSTM, last	256	0.556 [0.522, 0.587]	0.512	68.4
Transformer, last	128	0.537 [0.503, 0.570]	0.472	73.9
RNN, last	256	0.534 [0.501, 0.568]	0.467	74.6
LSTM, mean	256	0.602 [0.569, 0.633]	0.598	68.4
Transformer, mean	128	0.632 [0.599, 0.663]	0.625	73.9
RNN, mean	256	0.622 [0.588, 0.654]	0.611	74.6
LSTM + frozen SGNS, mean	256	0.630 [0.597, 0.661]	0.626	71.3
LSTM + frozen GloVe, mean	256	**0.655** [0.623, 0.687]	0.651	73.7
GloVe average (no LM)	*100*	*0.731* [0.697, 0.758]	*0.730*	–

Pooling.

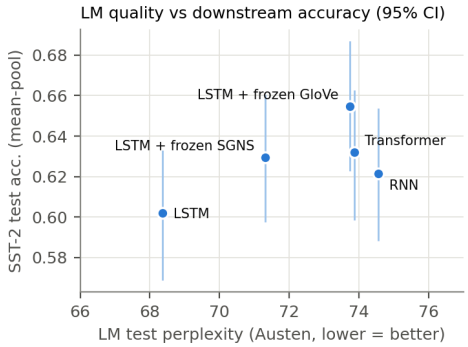
- Mean pooling beats last-state pooling for every model. The LSTM's gain is +4.6 points, with a paired 95% CI of [+0.7, +8.4]. Last-state features are close to chance.
- *Hypothesis:* almost every SST-2 sentence ends in "." or a similar mark. At that position an LM's state encodes what is needed to predict the *next sentence*'s first token, which is mostly generic punctuation context, not the sentence's sentiment.
- Last-state accuracy also drops on sentences with negation (*not, n't, never, …*; 22% of test sentences): 0.42–0.47, against 0.55–0.58 without negation. The mean-pooled features show no such drop.

Models and embeddings.

- The Transformer's mean-pooled features beat the LSTM's by +3.0 points, but the CI [−0.5, +6.5] includes zero.
- **The frozen-GloVe LSTM beats the random-embedding LSTM by +5.3 points** (CI [+1.6, +8.9]).
- The plain GloVe average beats every LM representation, by +7.6 points (CI [+4.1, +10.9]) over the best one.

Does LM ranking predict downstream ranking? No.

- The best LM, the LSTM at PPL 68.4, gives the **worst** mean-pooled features. The GloVe-LSTM, the worst of the three LSTMs at modelling Austen, gives the best.
- Across the five LMs, Spearman ρ between PPL and accuracy is +0.3 (p = 0.62, n = 5). If anything the relationship goes the wrong way, but it is not significant.



Error analysis (from the `downstream.json` written by the notebook).

- **Both models wrong (216 of 872).** These are dominated by sentences whose sentiment-bearing words are `<unk>`:
  - "*sticky sweet sentimentality , clumsy plotting …*" is seen as `<unk> sweet <unk> , <unk> <unk> ….`
  - In "*it 's made with deftly unsettling genre flair*", every content word is unknown.
  - Contrastive sentences fail too. "*…takes an admirable look at … but it does so with such an uneven tone…*" keeps *admirable* but loses *uneven*, so it is predicted positive.
- **Unknown words matter, modestly.** Misclassified sentences have more `<unk>` tokens: 26.9% vs 25.5% for LSTM-mean.
- **Disagreements between models.** The Transformer alone gets comparative and ironic sentences right, such as "*i 'll bet the video game is a lot more fun than the film*" (*fun* and *film* are both `<unk>`). The cases only the LSTM gets right are short or low-OOV sentences, such as "*cool ?*".
- **Bag-of-vectors and negation.** The GloVe average does lose accuracy on negated sentences (0.684 vs 0.744). Sequence features do not, but they are weaker overall.

5. Discussion, limitations and lessons

Answer to the comparison question.

- For next-token prediction on a narrow corpus, what matters is the architecture and its ability to adapt all parameters to the corpus. The LSTM beats the RNN and the Transformer, trainable embeddings beat frozen ones, and in-domain fixed embeddings beat 6B-token GloVe.
- For transfer to a different domain and task, the ranking reverses. General-English knowledge in the frozen GloVe table is worth more than anything the LMs learned from 340k tokens of Austen. The model that fits Austen best transfers worst.
- **Part II explains Part III.** Trainable embeddings specialise to Austen's distribution, which helps perplexity and hurts transfer. Frozen GloVe rows keep word meanings such as polarity (*admirable, bad*) that the classifier can still read through the LSTM. This is a hypothesis consistent with the results, not something I tested directly.
- So better perplexity did not mean better representations in this experiment.

Limitations.

- **One seed per configuration.** PPL differences under about 1 point (RNN vs Transformer) should not be read as rankings.
- **Limited tuning.** The learning rate was tuned over only two values. The models do not have matched parameter counts, and they see different test-time contexts (carried state vs a 64-token window).
- **Noisy GPU timings** on the laptop: up to 3.7× spread across repeated passes. CPU throughput is more stable, and the training times are indicative only.
- **Domain mismatch dominates Part III.** With 26% `<unk>`, the absolute accuracies say more about vocabulary coverage than about the architectures.
- **Train/test format mismatch.** SST-2 training uses phrases (≥ 5 words) while the test set uses full sentences.
- **Implementation change.** The LSTM learning-rate sweep used the fused cuDNN layer, before I switched to stacked layers to avoid the exit crash. It gave 66.5 vs 66.9 validation PPL, so the two are equivalent. The sweep outcomes are recorded in the notebook, and `RERUN_LR_SWEEP` repeats them.

Lessons.

1. Tune at least the learning rate per architecture. A shared lr of 1e-3 initially ranked the LSTM last.
2. Report uncertainty. Bootstrap CIs showed that two of the five Part III differences I first wanted to interpret are not significant.
3. Pick downstream data with the LM's vocabulary in mind, or use subword tokenisation such as BPE.
4. Validate timing measurements against a control. The three identical LSTMs exposed the GPU timing noise.

References and tool use

- Project Gutenberg texts via NLTK (Bird et al., 2009). Kneser & Ney (1995); Chen & Goodman (1998).
- Mikolov et al. (2013), word2vec, implemented in gensim (Řehůřek & Sojka, 2010). Pennington et al. (2014), GloVe 6B, obtained via gensim-data.
- Socher et al. (2013), SST, and Wang et al. (2019), GLUE, via HuggingFace datasets. PyTorch, scikit-learn.

- **AI assistance.** Claude Code (Anthropic) was used as a coding assistant and for debugging, and to draft this report based on my findings.